

System Requirements Specification: Hospital Smart Queue System

Project Lead: ياسين مصطفى وهب

3. Requirements Gathering

This section details the systemic and operational requirements for the Hospital Smart Queue System. The proposed system architecture replaces manual queueing with a centralized, automated digital infrastructure.

3.1 Stakeholder Analysis

Identifying the primary stakeholders and their functional needs is necessary for establishing the system's role-based access control methodology.

Stakeholder	Functional Needs & Responsibilities
Administrators	Require systemic oversight and data management capabilities. Their needs include managing institutional staff credentials and accessing analytical data regarding system utilization.
Receptionists	Function as the primary operational controllers of the daily schedule. They require interfaces that permit the immediate creation, modification, and cancellation of patient appointments across all hospital departments.
Patients	Represent the end-users seeking medical services. They require an accessible client-side interface to

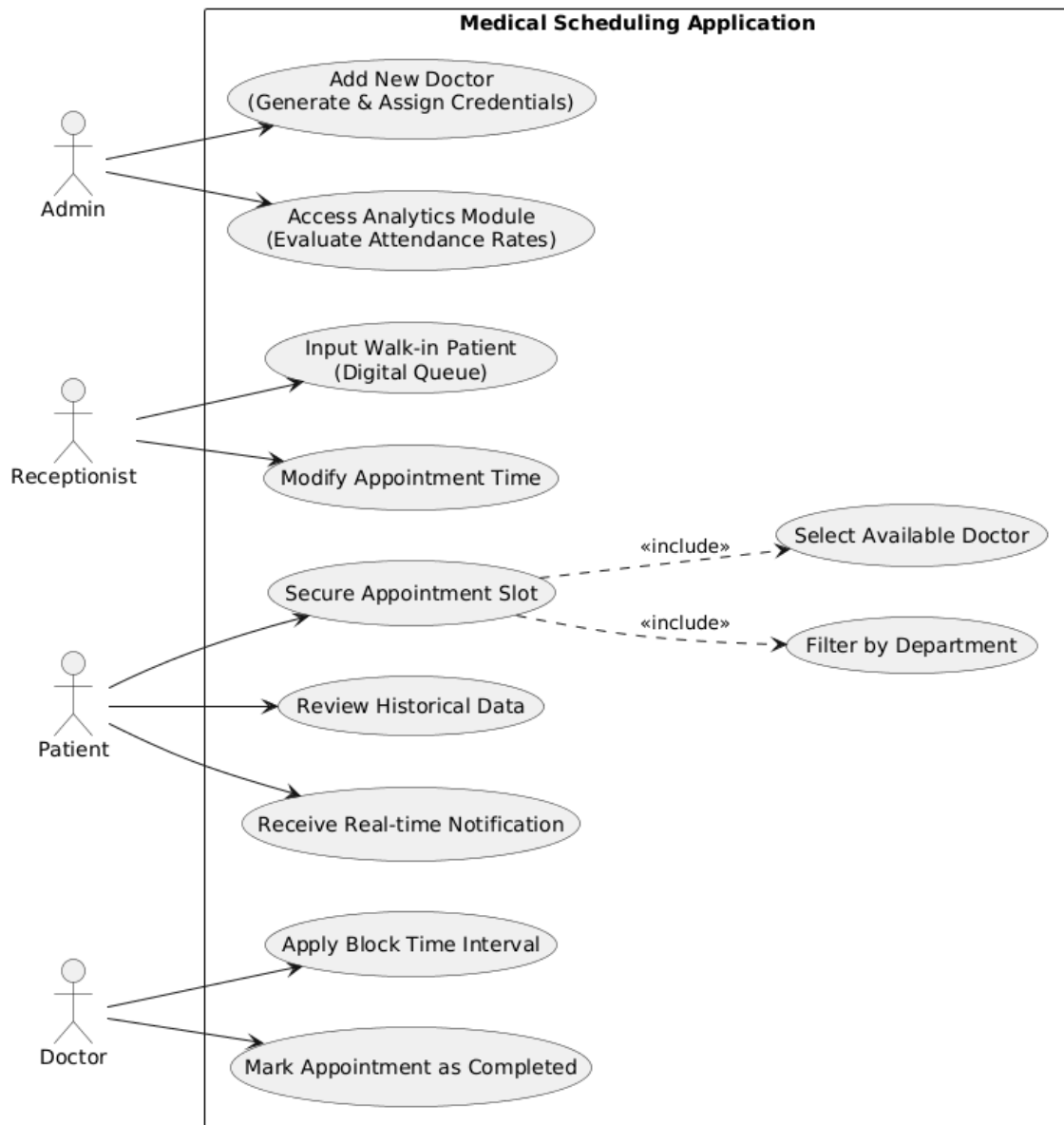
Stakeholder	Functional Needs & Responsibilities
	browse departmental availability, self-schedule appointments, and receive automated scheduling alerts.
Doctors	Act as the service providers within the architecture. They need localized dashboard views to monitor their specific daily schedules, update consultation statuses, and dictate their temporal availability.

3.2 User Stories & Use Cases

The following scenarios demonstrate how the defined stakeholders interact with the application architecture.

- **Admin Scenarios**
 - An administrator adds a new doctor to the system by generating and assigning specific email credentials and passwords.
 - An administrator accesses the analytics module to evaluate weekly appointment volume against actual patient attendance rates.
- **Receptionist Scenarios**
 - A receptionist inputs a walk-in patient into the digital queue or modifies an existing appointment time due to scheduling conflicts.
- **Patient Scenarios**
 - A patient filters the institutional database by department, selects an available doctor, and secures an appointment slot.
 - A patient reviews their historical appointment data and receives a real-time notification one hour prior to an upcoming consultation.
- **Doctor Scenarios**
 - A doctor applies a "Block Time" interval to their schedule to reflect unavailability during a surgical procedure, successfully preventing patient bookings for that duration.

- Upon concluding a consultation, a doctor marks the corresponding appointment as "Completed" to update the central database and analytics engine.



3.3 Functional Requirements

The system develops specific features distributed across distinct modular components.

1. **Authentication Module:** The system must execute secure user registration and login protocols utilizing JSON Web Tokens (JWT) for session management across all four user roles.
2. **Administrative Module:** The backend infrastructure must support Create, Read, Update, and Delete (CRUD) operations for user profiles and hospital departments. The system must also generate visual statistical representations using chart libraries.
3. **Scheduling Module:** The database schema must process appointment data, ensuring that requested time slots do not conflict with existing bookings or doctor-defined block times. Receptionists require global CRUD access to all appointment objects.
4. **Real-time Communication Module:** The architecture must integrate Socket.IO to push live updates across dashboards and transmit synchronous notification alerts to patients.

3.4 Non-functional Requirements

The application infrastructure must satisfy the following technical and qualitative criteria.

- **Performance:** The system must process concurrent user requests without latency, particularly during real-time schedule updates and automated notifications managed by Socket.IO.
 - **Security:** Implementation of JWT authentication is required to maintain strict data segregation, ensuring that users can only access data and functions explicitly authorized for their role.
 - **Usability:** Client-side interfaces, constructed with React.js and Tailwind CSS, must provide distinct, intuitive navigation paths tailored to the specific operational workflows of admins, doctors, receptionists, and patients.
 - **Reliability:** The Node.js and Express.js backend framework, connected to a MongoDB continuous database, must maintain high availability and accurate data persistence to prevent scheduling errors in a clinical environment.
-